

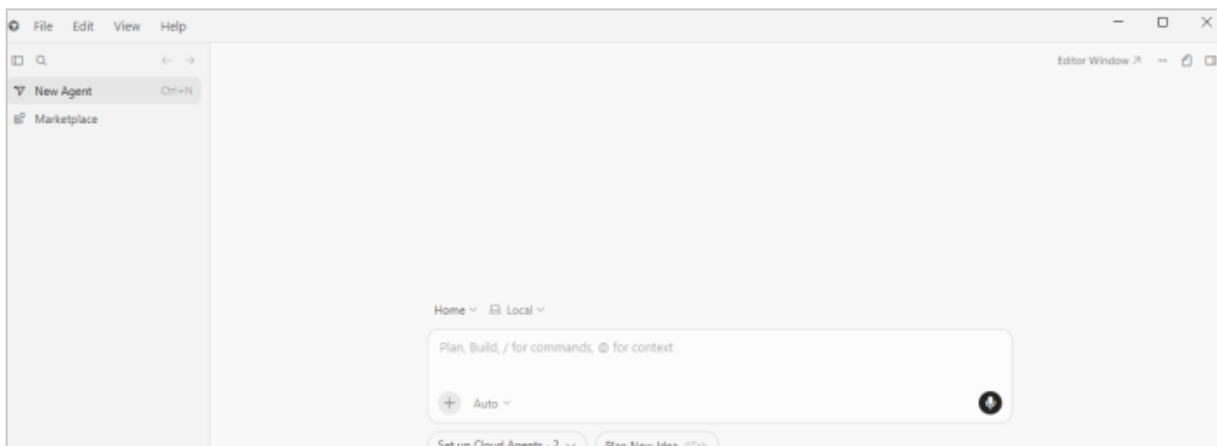
GUIA DE INSTALAÇÃO

Setup do Ambiente no Linux

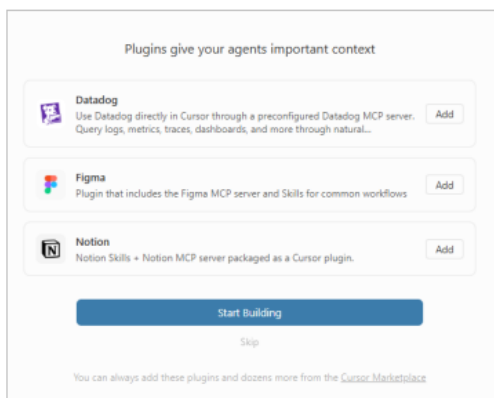
Este guia descreve, passo a passo, a instalação e configuração de todas as ferramentas necessárias para o ambiente de desenvolvimento no Linux (Ubuntu/Debian). Siga os passos na ordem indicada.

Passo 1 Instalar o Cursor

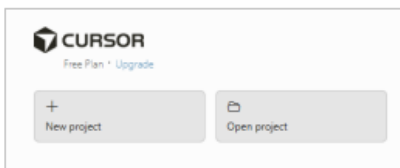
- ▶ Acesse <https://cursor.com>
- ▶ Clique em **Download for Linux**, baixe e instale o arquivo
- ▶ **Abra o Cursor e faça login** com sua conta. Crie uma se ainda não tiver (**use o email institucional do IPOG**).
- ▶ Após o login, a tela de chat será aberta. Troque para **Editor Windows** (opção no canto superior direito).



- ▶ Você pode escolher adicionar os 3 plugins e depois clicar em **Start Building**.



- ▶ Selecione **New Project**.



Passo 2 Instalar o Git

- ▶ No Terminal, execute:

```
sudo apt install -y git
git --version
```

Deve aparecer algo como **git version 2.43.0**.

Passo 3 Instalar Node.js e npm

- ▶ No Terminal, execute:

```
sudo apt install -y nodejs npm
node -v
npm -v
```

Passo 4 Instalar Python 3

- ▶ No Terminal, execute:

```
sudo apt install -y python3-full python3-venv python3-pip
python3 --version
```

Passo 5 Instalação das Ferramentas

- ▶ No Terminal, crie uma pasta para o projeto e entre nela:

```
mkdir hackathon
cd hackathon
```

- ▶ Abra o Cursor e entre na pasta do projeto: hackathon.
- ▶ Abra o Terminal dentro do Cursor (**Terminal > New Terminal**).
- ▶ Crie o venv na pasta atual:

```
python3 -m venv venv
```

- ▶ Ative o venv (note o (venv) que aparece no início do prompt):

```
source venv/bin/activate  
pip install --upgrade pip
```

Passo 6 Instalar Jupyter Lab e dependências

- ▶ Criar e ativar o venv:

```
python3 -m venv venv  
source venv/bin/activate
```

- ▶ Atualizar o pip:

```
pip install --upgrade pip
```

- ▶ Instalar o uv (gerenciador rápido de pacotes Python):

```
pip install uv
```

- ▶ Instalar Jupyter Lab e ferramentas necessárias:

```
pip install jupyterlab==4.4.1 jupyter-collaboration==4.0.2 ipykernel  
pip install datalayer_pycrdt==0.12.17
```

- ▶ Para iniciar o Jupyter Lab:

```
jupyter lab --port 8888 --IdentityProvider.token MY_TOKEN --ServerApp.root_dir ./
```

💡 **O que é esse MY_TOKEN?** É uma senha local que protege seu Jupyter Lab. Para o bootcamp, pode usar algo simples como bootcamp123. Em projetos reais, sempre gere algo aleatório — e nunca suba esse token pro Git.

⚠ **Importante:** o Jupyter Lab fica rodando enquanto esse terminal estiver aberto — não feche! Abra outro terminal para outras coisas.

Passo 7 Configurar os MCPs

- ▶ Abra o Terminal no próprio Cursor, crie o ambiente virtual e o ative:

```
python3 -m venv venv
source venv/bin/activate
```

MCP Context7

- ▶ Abra a pasta do projeto no Cursor e, no Terminal do Cursor, faça login no Context7 (vai abrir um navegador — logue com o email institucional). Os alunos precisam poder fazer isso com seus emails pessoais.

```
npx -y ctx7 login
```

Vai aparecer uma tela para fazer login ou criar a conta (signup) e depois dar permissão de acesso.

- ▶ Configure no Cursor:

```
npx ctx7 setup --cursor -y
```

- ▶ Reinicie o Cursor (feche completamente e abra novamente).
- ▶ Reative o ambiente virtual:

```
source venv/bin/activate
```

Para testar, no chat do Cursor digite:

Use o MCP Context7 para explicar boas práticas de APIs REST em Spring Boot

Se a resposta vier com referências reais à documentação atual, está funcionando.

MCP Jupyter — executar células de notebook

- ▶ Crie a pasta `.cursor` na raiz do seu projeto e, dentro dela, um arquivo `mcp.json`:

```
mkdir -p .cursor
nano .cursor/mcp.json
```

- ▶ Cole o conteúdo:

```
{
  "mcpServers": {
    "jupyter": {
      "command": "uvx",
      "args": ["jupyter-mcp-server@latest"],
      "env": {
        "JUPYTER_URL": "http://localhost:8888",
        "JUPYTER_TOKEN": "MY_TOKEN",
        "ALLOW_IMG_OUTPUT": "true"
      }
    }
  }
}
```

```

    }
  }
}

```

 **Substitua** MY_TOKEN pelo token que você definiu ao iniciar o Jupyter Lab.

Passo 8 MCP Filesystem

O que faz: permite à IA listar e ler arquivos do seu projeto.

⚠ Princípio de segurança: você define qual pasta a IA pode acessar. NÃO dê acesso ao seu ~/.ssh ou à raiz do disco — só à pasta do projeto.

► Instalar globalmente:

```
npm install -g @modelcontextprotocol/server-fileSystem
```

► Adicione ao mcp.json (junte com a configuração do Jupyter):

```

{
  "mcpServers": {
    "jupyter": {
      "command": "uvx",
      "args": ["jupyter-mcp-server@latest"],
      "env": {
        "JUPYTER_URL": "http://localhost:8888",
        "JUPYTER_TOKEN": "MY_TOKEN",
        "ALLOW_IMG_OUTPUT": "true"
      }
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-fileSystem",
        "/caminho/completo/para/seu/projeto"
      ]
    }
  }
}

```

💡 **Substitua** /caminho/completo/para/seu/projeto pelo caminho da pasta do seu projeto.
Exemplo: /home/joao/Projetos/hackathon

Passo 9 Ativar os MCPs no Cursor

Depois de salvar o `mcp.json`:

- ▶ **Feche o Cursor completamente** (não basta fechar a janela — feche o programa todo).
- ▶ Abra o Cursor novamente.
- ▶ Reative o ambiente virtual: `source venv/bin/activate`
- ▶ Vá em **File → Preferences → Cursor Settings → Tools and MCP**.
- ▶ Marque os MCPs que você configurou (**filesystem, jupyter, context7**).
- ▶ **Reinicie o Cursor novamente.**

Para testar o **filesystem**, no chat do Cursor:

Use o MCP filesystem para listar todos os arquivos do meu projeto

Para testar o **Jupyter**, com o Jupyter Lab rodando, no chat:

Abra o notebook `01_exploracao.ipynb`, crie uma célula que importa pandas e executa `print("Olá!")`. Me diga exatamente o que apareceu.

✓ Verificação final — checklist

Antes do bootcamp começar, confira que você tem todos esses comandos funcionando:

```
git --version
node -v
npm -v
python3 --version
docker --version
source venv/bin/activate
jupyter --version
```

MCPs (dentro do Cursor)

Em Settings → Tools and MCP, devem aparecer:

- ▶  **context7** (verde)

- ▶ ☒ jupyter (verde)
- ▶ ☒ filesystem (verde)